

Chapter 5: Structured (Non-Scalar) Data Types

EEC 3150

Trevecca Nazarene University

Outline

- 1 Tuples
- 2 Lists and Mutability
- 3 Strings
- 4 Dictionaries
- 5 Sets

Tuples

Structured and unstructured data types:

Unstructured data types (or scalar dtypes) are objects which have no internal structure (they represent one piece of information)

- Examples: int, float, bool

Structured data types (or non-scalar dtypes) are objects which do have some kind of internal structure (they can represent multiple pieces of information)

- Examples we've seen already: string, range, tuple
- New examples: lists, dictionaries, sets

Tuples

Defn: tuples

Tuple: an ordered, immutable sequence of data elements

Rules:

- Parentheses, comma-separated
- Index/slice with square brackets (just like strings)

Examples:

- `x = (1,)` (singletons require trailing comma)
- `x = (1,2,3)`
- `x = ()` (empty tuple)
- `x = (1, 'cow', True)` (multiple data types allowed)
- `x[0]` (index the 0th element of a tuple, like strings)
- `x = ( (1,2), (3,4) )` (tuple of tuples)

Tuples

Tuples in action:

Operations on tuples:

- Concatenation: `+` (like strings)
- Repetition: `*` (like strings)
- Length: `len()` function
- Casting: `tuple()`

Uses:

- Multiple arguments to a function
- Multiple returns from a function
- Shape of multi-dimensional numpy arrays (more on that later)
- Iterable with a for loop

Lists and Mutability

Defn: lists

List: an ordered, mutable sequence of data elements

Rules:

- Square brackets, comma-separated
- Index/slice with square brackets (just like strings)

Examples:

- `x = [1]` (singletons don't need comma)
- `x = [1,2,3]`
- `x = []` (empty list)
- `x = [1, 'cow', True]` (multiple data types allowed)
- `x[0]` (index the 0th element of a list, like strings)
- `x = [ [1,2], [3,4] ]` (list of lists)

Lists in action:

Operations on lists:

- Concatenation: `+` (like strings)
- Repetition: `*` (like strings)
- Length: `len()` function
- Casting: `list()`

Uses:

- Used infinitely more than tuples
- Let's talk about mutability first

Mutability

Let's compare these codes:

- These codes demonstrate defining a tuple/list y in terms of another x and then changing x afterwards
- In which codes does changing x change y ?
(`id()` returns an object's location in memory)

```
x = (1,)
y = x
print(y, x)
print( id(y), id(x) )
```

```
x = [1]
y = x
print(y, x)
print( id(y), id(x) )
```

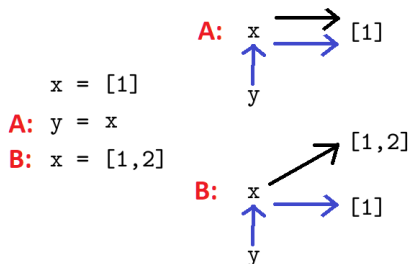
```
x = (1,2)
# x += (2,)

print(y, x)
print( id(y), id(x) )
```

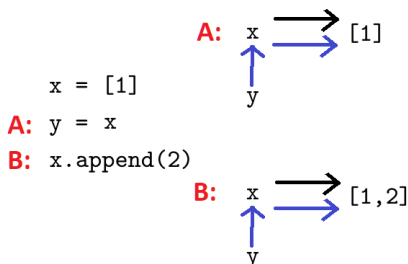
```
x = [1,2]
# x += [2]
# x.append(2)

print(y, x)
print( id(y), id(x) )
```

Mutability



- A: pass the current memory location of x to y
- B: change the contents of x (this method also changes it's location in memory)
- B: y still points to x's old location, so it's unchanged



- A: pass the current memory location of x to y
- B: change the contents of x (this method preserves it's location in memory)
- B: x, y still point to orig. locations, but with new values

Mutability

Defn: mutability

Mutable objects in Python (and other languages) can be altered after being defined (opposite: immutable)

Consequences:

- Convenient for manipulating data (MANY cool functions)
- User-defined functions can alter global mutable objects WITHOUT the global keyword (show example code)
- Possible to unintentionally alter objects when defining them in terms of each other or copying (like the previous example)
- Solution: deep copy instead of shallow copy

Examples:

- Mutable: lists, dictionaries, sets
- Immutable: ints, floats, bools, strings, ranges, tuples, numpy arrays, pandas dataframes/series

List methods:

- **Methods** are functions that are defined for only certain types of objects
- They are called using *dot notation*:

```
x = [1]  
x.append(2)
```
- Go to this link to see Python's list methods: [LINK](#)
- Examples in my Python example scripts

Iterables:

Lists are **iterable**:

- Their elements can be returned one at a time and utilized in a for loop

```
x = [1, 'cow', True]
for i in x:
    print(x)
```

Building lists with a loop and append:

Lists are often constructed in this format:

- Start with an initial list (usually empty)
- Append in a loop

```
x = []
for i in range(10):
    if i%2 == 0:
        x.append(i)
```

Some simple list programs:

Write a function which squares all the elements of a list:

```
def SquareList(x):  
    y = []  
    for i in x:  
        y.append(i**2)  
    # end  
    return y
```

Write a function which will concatenate a list of strings into one string with spaces in between them (no trailing whitespace):

```
def ConcatStrings(x):  
    y = x[0]  
    for s in x[1:]:  
        y += ' '  
        y += s  
    # end  
    return y
```

Iterate through multiple lists: `zip()` and `enumerate()`:

Use `zip()` to iterate through multiple lists at once:

```
x = [1,2,3]
y = [4,5,6]
for i,j in zip(x,y):
    print(i,j)
```

- If lists are different lengths, it goes the length of the shorter one

Use `enumerate()` to iterate a list AND and its indices:

```
y = [4,5,6]
for i,j in enumerate(y):
    print(i,j)
```

- Useful when you want to iterate a list without indexing a constantly but you also want the indices

Applying a function to list elements: `map()`

- The `map(f,iter)` function applies the function `f` to each element in the iterable `iter` (list, tuple, etc.)
- It returns a map object which is useless so cast the result to the desired iterable at the end

```
def square(n):  
    return n**2  
# end  
x = [1,2,3]  
y = list(map(square,x))  
print(y)
```

You can use `map()` to cast the elements of a list in one line:

```
x = [1,2,3]  
y = list(map(str,x))  
print(y)
```

Primes:

Write a function which generates a list of the first n primes.

Be efficient: we only need to check if a number is divisible by any primes.

Nested lists:

It is very common in Python to make a list of lists:

```
>>> x = [ [1,2] , [3,4] ]
>>> x[0]
[1,2]
>>> x[0][0]
1
>>> x[1]
[3,4]
>>> x[1][1]
4
```

- To access the inner-most items of a nested list, you must index the list multiple times
- The first level of indexing returns one of the lists
- The second will then index THAT list

Terminology:

- 2D list: list of lists
- 3D list: list of lists of lists

Creating nested lists:

We can use nested loops to create/index nested lists:

```
x = []  
for i in range(n):  
    x.append([])  
    for j in range(n):  
        x[i].append( stuff )
```

- Start with an empty list `x`
- Outer loop: append empty lists to `x`
- Inner loop: append to the elements of `x` (which are themselves lists)

Terminology:

- Column: entries of a 2D list indexed vertically
- Row: entries of a 2D list indexed horizontally

Row sum:

Write a function takes a 2D list and returns two lists: the row sums and column sums

```
def RowColSum(x):  
    row = []  
    for y in x:  
        row.append(sum(y))  
    # end  
  
    n = len(x)  
    m = len(x[0])  
    col = []  
    for i in range(n):  
        col.append(0)  
        for j in range(m):  
            col[i] += x[j][i]  
        # end  
    # end  
  
    return row, col  
# end
```

- Notice how much easier it is to get the row sums than the column sums

Identity matrix:

Write a function which generates an n -dimension identity matrix using a nested lists.

Transpose:

Write a function which return the matrix transpose of a 2D list.

Strings

Strings revisited:

String: an ordered, immutable sequence of characters

- Go to this link to see Python's string methods: [LINK](#)
- Examples in my Python example scripts

ASCII:

ASCII: American Standard Code for Information Interchange

- All characters are stored as numbers
- An ASCII table shows these number/character conversion
- Conversion functions in Python:

```
>>> ord('p')
```

```
112
```

```
>>> chr(112)
```

```
'p'
```

Strings

Decimal	Binary	ASCII	Decimal	Binary	ASCII	Decimal	Binary	ASCII	Decimal	Binary	ASCII
0	00000000	NUL	32	00100000	SP	64	01000000	@	96	01100000	`
1	00000001	SOH	33	00100001	!	65	01000001	A	97	01100001	a
2	00000010	STX	34	00100010	"	66	01000010	B	98	01100010	b
3	00000011	ETX	35	00100011	#	67	01000011	C	99	01100011	c
4	00000100	EOT	36	00100100	\$	68	01000100	D	100	01100100	d
5	00000101	ENQ	37	00100101	%	69	01000101	E	101	01100101	e
6	00000110	ACK	38	00100110	&	70	01000110	F	102	01100110	f
7	00000111	BEL	39	00100111	'	71	01000111	G	103	01100111	g
8	00001000	BS	40	00101000	(	72	01001000	H	104	01101000	h
9	00001001	HT	41	00101001	)	73	01001001	I	105	01101001	i
10	00001010	LF	42	00101010	*	74	01001010	J	106	01101010	j
11	00001011	VT	43	00101011	+	75	01001011	K	107	01101011	k
12	00001100	FF	44	00101100	,	76	01001100	L	108	01101100	l
13	00001101	CR	45	00101101	-	77	01001101	M	109	01101101	m
14	00001110	SO	46	00101110	.	78	01001110	N	110	01101110	n
15	00001111	SI	47	00101111	/	79	01001111	O	111	01101111	o
16	00010000	DLE	48	00110000	0	80	01010000	P	112	01110000	p
17	00010001	DC1	49	00110001	1	81	01010001	Q	113	01110001	q
18	00010010	DC2	50	00110010	2	82	01010010	R	114	01110010	r
19	00010011	DC3	51	00110011	3	83	01010011	S	115	01110011	s
20	00010100	DC4	52	00110100	4	84	01010100	T	116	01110100	t
21	00010101	NAK	53	00110101	5	85	01010101	U	117	01110101	u
22	00010110	SYN	54	00110110	6	86	01010110	V	118	01110110	v
23	00010111	ETB	55	00110111	7	87	01010111	W	119	01110111	w
24	00011000	CAN	56	00111000	8	88	01011000	X	120	01111000	x
25	00011001	EM	57	00111001	9	89	01011001	Y	121	01111001	y
26	00011010	SUB	58	00111010	:	90	01011010	Z	122	01111010	z
27	00011011	ESC	59	00111011	;	91	01011011	[	123	01111011	{
28	00011100	FS	60	00111100	<	92	01011100	\	124	01111100	
29	00011101	GS	61	00111101	=	93	01011101	]	125	01111101	}
30	00011110	RS	62	00111110	>	94	01011110	^	126	01111110	~
31	00011111	US	63	00111111	?	95	01011111	_	127	01111111	DEL

Strings

More on strings:

- Sorting a list of strings will sort them numerically via their ASCII codes

Escape characters:

An escape character utilizes a backslash (`\`) to use characters in strings that would otherwise be impossible or illegal to add:

- `\n`: new line
- `\t`: tab
- `\'`: single quote
- `\"`: double quote
- `\\`: backslash

See dictionaries:

Our in-class exercise for strings will also use dictionaries. Look for it in the next section.

Dictionaries

Defn: dictionaries

Dictionaries: an ordered (as of 3.7), mutable sequence of data elements (values) which are indexed with arbitrary labels (keys)

Rules:

- Curly brackets, comma-separated
- Index/slice with square brackets (just like strings)
- Access a *value* by indexing with a *key*

Examples:

- `x = { 'a':1, 'b':2, 'c':3, 1234:True} ( key:value )`
- `>>> x['a']`
1
- `>>> x[1234]`
True

Dictionaries

Building dictionaries:

```
x = { }  
x['a'] = 1  
x[5] = 'hello'
```

- You can build dictionaries by appending to an empty dictionary via indexing

A common application for dictionaries:

```
person = {  
    'firstname': 'Graham'  
    'lastname': 'West'  
    'age': 28  
    'employer': 'TNU'  
    'job': 'instructor'  
    'courses': ['EEC3150', 'INT0960']  
}
```

- You can use dictionaries as a database to store multiple kinds of data about one thing

Dictionaries

Methods: `x.keys()` and `x.values()`

Here are two critical dictionary methods:

```
>>> x = { 'a':1, 'b':2, 'c':3 }
>>> x.keys()
dict_keys(['a', 'b', 'c'])
>>> x.values()
dict_values([1, 2, 3])
```

- `x.keys()` will return the keys of `x`
- `x.values()` will return the values of `x`
- These aren't lists, but they are iterable

Iterating over a dictionary with `x.items()`

```
x = { 'a':1, 'b':2, 'c':3 }
for k, v in x.items():
    print(k,v)
```

- The `x.items()` method allows you to iterate over key/values pairs
- There are more dictionary methods, but these are the most important

String character counts:

- Write functions which will return a dictionary containing the counts of every unique character/word in a string
- Ignore case in both functions
- In the word function, strip any whitespace or punctuation
- Print each in descending order

Sets